CompanyOS — Complete Project Handbook

Version: 2026-11-04. Owner: Siddharth Sahoo. Repository: github.com/sidsahoo1203/CompanyOS

0. How to use this handbook

This single file records everything about CompanyOS: what it is, why it exists, how to use it, how it is built, every piece of work done from the first day to today, the current deployment, the security findings, and what is still open. It replaces the chat history, which will be deleted.

To continue work with Claude Code (or any AI assistant) later:

1. Open the project folder `C:\Users\91635\OneDrive\Desktop\CODE\COMPANYOS` in Claude Code (or share the GitHub repository).
2. Paste or attach this handbook (the `.md`, `.docx` or `.pdf` version).
3. Start with a message like the one in Section 14 ("Starter prompt for a new session").

The repository also contains detailed source documents that this handbook summarizes: `README.md`, `AGENTS.md` (rules for AI agents), `docs/phase-0.md` (requirements and architecture), `docs/milestone-N-plan.md` and `docs/milestone-N-verification.md`, `docs/production-readiness.md`, `docs/unit-testing.md`, `docs/integration-testing.md` and `docs/deployment-render.md`.

1. Quick facts

Item	Value
Product	CompanyOS: an approval-driven operations workspace for small digital agencies
Target users	5–50 person agencies and small startups; buyer is the founder or operations lead
Project folder (only location)	`C:\Users\91635\OneDrive\Desktop\CODE\COMPANYOS`
Old folder (deleted, do not use)	`C:\Users\91635\Documents\Codex\2026-10-04\you-are-my-autonomous-senior-product\outputs\companyos`
GitHub repository	`https://github.com/sidsahoo1203/CompanyOS` (branch `main`)
Live deployment	Render web service, address `https://companyos-bxk6.onrender.com` (free plan, Singapore)
Database (live)	Render PostgreSQL `companyos-db` (free plan; expires after about 30 days unless upgraded)
Email provider	Resend (sender `CompanyOS <onboarding@resend.dev>` until a domain is bought)
Stack	Next.js 16.4.3 (App Router), React 19, strict TypeScript, PostgreSQL 18, Better Auth 1.7.7, Zod, `pg`
Local database	Embedded PostgreSQL inside the project (`work/postgres`), port 55432
Status	Milestones 1–4 complete; production mode added; deployed to Render for testing
Tests	439 unit tests, 89 PostgreSQL integration tests, Playwright browser tests; all passing at last run
Security audit reports	`C:\Users\91635\security-audit-skill\companyos\run-1\` (outside the project)

2. What CompanyOS is and why it exists

2.1 The problem

Small agencies keep their know-how (onboarding steps, client rules, templates, pricing notes) scattered across documents and people's heads. When work is delegated, mistakes happen: wrong information is sent to clients, nobody knows who approved what, and new staff cannot find the right procedure.

AI tools promise to automate this, but they are risky for a business: they can invent facts, act without permission, and leave no record of who decided what.

2.2 The idea

CompanyOS is a shared workspace where an agency:

1. **Stores its knowledge** (documents) with clear access rules.
2. **Finds answers with citations**: every answer quotes the exact passage from the exact document, or says there is not enough information.
3. **Writes procedures** (step-by-step instructions linked to the knowledge they depend on).
4. **Turns a procedure into a "skill"** that drafts work, such as a client message, from the authorized sources.
5. **Requires a human approval** before anything is "sent". The approver sees the exact recipient, content, sources and a fingerprint, and every decision is recorded.

The core principle: **the software proposes, a person decides, and everything is recorded.** Sourced facts, suggestions, proposed actions and completed actions are always kept separate.

2.3 Why use it (benefits)

- One trusted place for company knowledge, with per-source privacy (everyone, admins only, or the creator only).
- Answers you can verify, with links back to the source passage; no invented answers.
- Repeatable procedures with version history; old versions are never overwritten.
- A safety gate: nothing goes out without an owner or admin approving the exact final content.
- A full audit trail of who requested, edited, approved or rejected what, and when.
- Strong data isolation: one company can never see another company's data, enforced in both the application and the database.

2.4 What it is not (yet)

- It does not call ChatGPT, Claude or any AI model. The drafting engine (`local-excerpts-v1`) assembles approved excerpts deterministically and costs nothing.
- It does not send real emails, Slack messages or files to clients. Approved actions are recorded in a "local outbox" (a database table). Real delivery (Gmail, Slack, Drive) is the planned Milestone 5.
- No billing, account deletion or data export yet.

3. Features (what a user can do)

3.1 Accounts and workspaces (Milestone 1)

- Sign up, sign in, sign out; sessions stored in the database (7 days, refreshed daily).
- In production: email verification is required before first sign-in, plus "Forgot password?" reset by email.
- Create up to 10 workspaces per person; switch between them.
- Roles: **Owner** (full control), **Admin** (manage content, invite members), **Member** (read, search, run skills, request approvals).
- Invitations: an owner or admin creates an invitation link for a specific email and role. No email is sent for invitations; the link is shared manually. The invited person signs in with that exact email and opens the link. Links expire, work once, and only the hash is stored.
- Owner changes roles; owner can remove admins and members; admins can remove members; anyone can leave. The owner cannot be removed (no ownership transfer yet).
- Settings: workspace name, industry, timezone, language, retention preference (saved, not enforced yet).
- Members page and responsive dashboard.

3.2 Knowledge library and Assistant (Milestone 2)

- Owners and admins create **sources** (folders) with visibility: all members, owners/admins only, or creator only.
- Upload PDF, DOCX, TXT, CSV or Markdown (max 5 MB per file). Files are stored privately in PostgreSQL; there are no public download links.
- Uploading the same filename to the same source creates a new immutable **version**; search uses only the latest version.
- Processing extracts text and splits it into passages. Status: queued, processing, ready, failed or quarantined. Click "Process / retry" if needed.
- **Search and Assistant:** keyword search (PostgreSQL full text) over sources you are allowed to read. The Assistant quotes exact passages with citation links, or says knowledge is insufficient. Hindi and other Indian scripts are supported.
- Safety checks quarantine files with active content (JavaScript in PDFs, embedded files) or suspicious "instructions to the assistant". This is not antivirus.
- Limits: 50 sources, 100 document versions, 50 MB per workspace; 100 PDF pages; 200,000 characters per document.

3.3 Procedures, skills and runs (Milestone 3)

- Owners and admins publish **procedures**: instructions plus 1–8 required knowledge sources. Editing publishes a new version; old versions stay.
- A **skill** pins one exact procedure version, a draft type (source brief or message proposal), a token and cost ceiling, and a 1–15 second timeout. Skills can be enabled or disabled.
- Members run enabled skills whose sources they can read. A **run** goes through four recorded steps: Resolve sources → Draft → Validate output → Enforce policy.
- Output is checked against a strict schema; citations must come from authorized sources. Proposed messages are always marked "approval required, execution disabled".
- Failures (missing sources, timeouts, forged citations, budget exceeded) are stored with a reason and never retried automatically.
- Limits: 50 procedures, 50 versions each, 100 skills, 1,000 runs per workspace (lifetime), 20 runs per person per minute, 3 concurrent runs.

3.4 Approvals and local outbox (Milestone 4)

- From a run's message proposal, click **Request approval**. The request snapshots recipient, content, inputs, sources and expected effect, plus a SHA-256 fingerprint.
- **Approvals inbox:** owners and admins see requests whose sources they can read; members see only their own.
- To approve: tick "I reviewed…" and confirm the shown fingerprint. If anything changed in the meantime, the decision is refused as stale.
- Nobody approves their own content except the workspace owner (so one-person agencies can work).
- Edit before approval creates a new revision with a new fingerprint and a new 48-hour expiry; old revisions can never be approved.
- Approving writes the exact approved snapshot once to the local outbox (`app.action_execution`, executor `local-outbox-v1`). Replays and double approvals are refused.
- Every request, edit, decision, expiry, execution and refused attempt is audited (identifiers and reasons only, never content).
- Limits: 2,000 approval requests per workspace (lifetime), 30 decisions or edits per person per minute.

4. How to use CompanyOS (step by step)

4.1 First walkthrough

1. Open the site and choose **Create your workspace** (sign up). In production, open the verification email and click **Verify email**.
2. On onboarding, create a workspace (name, industry, timezone, language).
3. **Members**: invite teammates (copy the invitation link and send it yourself).
4. **Knowledge**: create a source, choose its visibility, upload a document, wait for "Ready" (or click Process / retry).
5. **Assistant**: ask a question; check the cited passages.
6. **Procedures**: publish a procedure that uses your source; create a skill from it and enable it.
7. Run the skill with an objective and inputs; click **Process draft**; review the steps, draft and citations.
8. On a message proposal, click **Request approval**.
9. **Approvals**: as owner or admin, open the request, review the exact content and fingerprint, tick the confirmation, and approve, reject or edit.
10. See the local-outbox record and the audit trail on the request.

4.2 Roles summary

Role	Read workspace	Edit settings	Invite members	Invite admins	Change roles	Remove people	Approve
Owner	Yes	Yes	Yes	Yes	Yes	Admins and members	Yes (including own content)
Admin	Yes	Yes	Yes	No	No	Members (and leave)	Yes (not own content)
Member	Yes	No	No	No	No	No (can leave)	No

5. Running the project locally (on the laptop)

5.1 Requirements

Node.js 22.16 or newer (tested on 24.19), npm, Git. No Docker and no separate PostgreSQL install are needed; the database is embedded in the project.

5.2 Commands

Open two terminals in `C:\Users\91635\OneDrive\Desktop\CODE\COMPANYOS`.

First time only:

```
npm ci
npm run db:local        (terminal 1; keep it open)
npm run db:migrate      (terminal 2)
npm run db:seed         (terminal 2; optional sample accounts)
npm run dev             (terminal 2)
```

Every other time: `npm run db:local` in terminal 1 and `npm run dev` in terminal 2. Open `http://127.0.0.1:3000`. Stop both with Ctrl+C.

5.3 Local accounts

The seed creates fictional accounts: `founder@studio.example` (owner of "Northstar Studio · Sample" and of the workspace "Siddharth Sahoo"), `admin@studio.example`, `member@studio.example`, `founder@other.example` (a separate company, to show isolation). All share the password stored as `SEED_PASSWORD` in `.env.local`. Show it with:

```
findstr SEED_PASSWORD .env.local
```

Passwords are stored hashed and cannot be recovered; only reset. Local accounts exist only on the laptop, not on Render.

In local mode no email is sent: password-reset links are printed in the `npm run dev` terminal as `email.local_preview` lines, and sign-up does not need verification.

5.4 Files that must never be shared or committed

`.env.local` (local secrets), `work/` (local database and its passwords in `work/local-db.json`), `test-results/`, `playwright-report/`. They are listed in `.gitignore`.

5.5 OneDrive warning

The project lives in a OneDrive-synced folder. OneDrive can lock database files while PostgreSQL writes them. If database errors appear, pause OneDrive syncing while the app runs. If `npm run db:local` fails because an old PostgreSQL is still running, stop it cleanly with:

```
node_modules\@embedded-postgres\windows-x64\native\bin\pg_ctl.exe stop -D work/postgres -m fast
```

6. Architecture and technical design

6.1 Overall shape

A **modular monolith**: one Next.js application contains both the pages (frontend) and the API (backend). It must be deployed as one service. Splitting the frontend (for example to Vercel) and the API (to Render) breaks sign-in cookies and the same-origin security checks, and the server-rendered pages read the database directly.

6.2 Folder map

Path	Contents
`src/app/`	Pages (App Router) and `api/` route handlers
`src/app/(auth)/`	Sign-in, sign-up, forgot-password, reset-password pages
`src/app/w/[workspaceId]/`	Workspace pages: dashboard, knowledge, assistant, procedures, runs, approvals, members, settings
`src/components/`	Client UI components (forms, panels, inbox)
`src/lib/`	Server-only services: auth, db, env, permissions, knowledge, agents, approvals, email, logging
`src/proxy.ts`	Security headers (CSP with nonce, HSTS in production)
`db/migrations/`	Numbered SQL migrations 001–010 (schema, row-level security, functions)
`scripts/`	Local database helper, migration runner, seed, parser, test server, Render start and role setup
`tests/unit`, `tests/integration`, `tests/e2e`	Unit (Vitest), PostgreSQL integration, Playwright browser tests
`docs/`	Requirements, plans, verification reports, testing, deployment, this handbook
`render.yaml`	Render Blueprint (web service + database)

6.3 Security design (how data is protected)

- **Authentication:** Better Auth, email and password (12–128 characters), salted hashes, database sessions, no cached roles. Password change and reset revoke other sessions.
- **Two database roles at runtime:** `companyos_auth` can only touch authentication tables; `companyos_app` has no superuser, no RLS bypass, no table ownership and no direct writes. The migration (owner) account is never used by the running app.
- **Row-level security (RLS):** every tenant table has `workspace_id`; each request sets the actor and workspace inside a transaction (`asActor`), and PostgreSQL filters rows.
- **Writes only through SECURITY DEFINER SQL functions** with a fixed `search_path` and explicit permission checks, in addition to checks in the TypeScript service layer.
- **Request protection:** same-origin `Origin` check, Fetch Metadata cross-site rejection, JSON-only bodies with size limits, strict Zod validation, NUL-character rejection.
- **Headers:** nonce-based Content Security Policy, frame denial, no-referrer, cross-origin isolation, HSTS in production.
- **Untrusted content:** documents and run output are rendered as plain text; never as HTML; originals are never served.
- **Document parsing:** runs in a separate Node process with no secrets, a 192 MB heap limit, a 15-second deadline, a 64 MB cap on decompressed PDF data, and at most 2 parsers at once per server.
- **Logging:** one redacted JSON line per unexpected error (no bodies, documents, emails or tokens); the user sees a reference that matches the log line.

6.4 Database migrations

File	Purpose
001_foundation.sql	Auth tables, workspaces, memberships, invitations, audit events, RLS, core functions
002_knowledge.sql	Sources, documents, versions, chunks, processing leases, search, limits
003_procedures_agents.sql	Procedures, versions, skills, runs, steps, usage, quotas
004_procedure_visibility.sql	Procedure visibility rules
005_procedure_parent_acl.sql	Procedure access follows source access
006_approvals.sql	Approval requests, revisions, decisions, local outbox, audit
007_ascii_text_safety.sql	Hidden/bidirectional character checks written with ASCII escapes
008_member_removal.sql	Removing members; their pending approvals expire, their invites end
009_workspace_people.sql	"Name (email)" identity for current and former members
010_integration_fixes.sql	Removed requesters' requests cannot be edited back to life; document-view audit deduplication

Rule: never edit an applied migration (checksums are enforced); add a new numbered file.

6.5 Environment variables

Local (`.env.local`, created by `npm run db:local`): `APP_MODE=local`, `BETTER_AUTH_URL=http://127.0.0.1:3000`, `BETTER_AUTH_SECRET`, `DATABASE_URL`, `AUTH_DATABASE_URL`, `MIGRATION_DATABASE_URL`, test equivalents, `SEED_PASSWORD`, `UPLOAD_REQUIRE_ANTIVIRUS=false`.

Production (set in Render, never in files):

Variable	Meaning
`APP_MODE`	`production`
`NODE_VERSION`	`24`
`BETTER_AUTH_URL`	Exact public address, `https://…` with no trailing slash (currently `https://companyos-bxk6.onrender.com`)
`BETTER_AUTH_SECRET`	32+ random characters
`MIGRATION_DATABASE_URL`	Render database Internal URL (owner account)
`APP_DB_PASSWORD`, `AUTH_DB_PASSWORD`	24+ random characters each; passwords for the two restricted roles
`RESEND_API_KEY`	Resend key starting with `re_`
`EMAIL_FROM`	Sender, e.g. `CompanyOS <onboarding@resend.dev>` or `CompanyOS <no-reply@yourdomain>`
`UPLOAD_REQUIRE_ANTIVIRUS`	`false` until a real scanner exists
`PORT`	Set by Render (10000)

The start script builds `DATABASE_URL` and `AUTH_DATABASE_URL` itself and removes the owner URL and role passwords from the web server's environment.

7. Complete history of the work (start to today)

7.1 Phase 0: discovery and plan (2026-10-04)

Empty folder; Node 24.19, npm 11.2, Git available; no Docker or PostgreSQL. Wrote the requirements, architecture proposal (modular monolith, Next.js, PostgreSQL with RLS, Better Auth, Zod), data model, a six-milestone plan, risks and mitigations (`docs/phase-0.md`). Decision: build locally first; no paid services; no deployment until approved.

Milestone plan: 1 Foundation → 2 Knowledge → 3 Procedures and skills → 4 Approvals → 5 Integrations (Drive, Slack, Gmail) → 6 Usage, monitoring, privacy, deployment.

7.2 Milestone 1: foundation

Accounts, durable sessions, workspaces (multiple per user), owner/admin/member roles with live permission checks, expiring email-bound invitations shared manually, member directory, settings, responsive dashboard, migrations, synthetic seed data, isolation and permission tests, minimal security events.

7.3 Milestone 2: knowledge library

Private sources with visibility scopes; uploads stored privately in PostgreSQL; immutable versions; processing with leases and states; basic content checks (not antivirus); PostgreSQL full-text search; Assistant answering with exact cited excerpts or "insufficient knowledge"; citation links re-checked for permission. Evidence in `docs/milestone-2-verification.md`.

7.4 Milestone 3: procedures, skills and draft runs

Versioned procedures linked to sources; pinned skills with contracts, budgets and timeouts; private draft runs with four recorded steps; local excerpt engine (no AI model, zero cost); strict output validation; safe failures without automatic retries; usage recording. Evidence in `docs/milestone-3-verification.md`.

7.5 Milestone 4: approvals

Approval inbox and detail pages, fingerprinted snapshots, approve/reject/edit with immutable revisions, stale-content refusal, idempotency and replay protection, single execution into the local outbox, full audit trail, decision rate limits, Fetch Metadata hardening. Evidence in `docs/milestone-4-verification.md`.

7.6 AGENTS.md

Created a repository-level `AGENTS.md` with purpose, directories, commands, conventions, security rules, data isolation, secrets, AI/untrusted content rules, side-effect rules, deployment restrictions and the definition of done. Any AI agent working on the repo should read it first.

7.7 Production-readiness review and fixes (2026-10-09)

A full review found 5 critical and several high items. Fixed without needing decisions: member removal (migration 008), approver identity spoofing via display names (migration 009, "Name (email)" labels), invisible failures (redacted structured logging with references), a real bug where every PDF failed in production builds (parser packaging), rotated local secrets after they were shown in a transcript, and an accessibility fix on the run page. Remaining items needed owner decisions (hosting, email, backups, scanner). See `docs/production-readiness.md`.

Also during this period: the Windows-1252 local databases were migrated to UTF-8 (old ones renamed, not deleted) so Indian scripts, the ₹ sign and emoji work; invisible characters accidentally written into source were replaced by migration 007 and ASCII escapes.

7.8 Unit testing (2026-10-23)

418 unit tests in 21 files, about 90% line coverage (services 99%, API routes 100%, components 96%). Mutation checks proved the tests catch deliberate breakage. Bugs found and fixed: Hindi/Indic search terms were discarded; a NUL character caused a 503 instead of 400; the auth route returned 503 for non-object JSON; the browser client showed raw parser errors. See `docs/unit-testing.md`.

7.9 Integration testing with test-driven development (2026-10-23)

Wrote new PostgreSQL tests first (88 tests in 8 files). Real bugs found and fixed in migration 010: an admin could revive a removed member's expired approval by editing it; every document view wrote an audit row (now deduplicated per 10 minutes). Browser sign-up helpers now wait out the sign-up rate limit. See `docs/integration-testing.md`.

7.10 Penetration test (security agent review)

Requested as "break all the features, don't fix without permission". Result: nothing critical exploitable on the laptop, but 26 weaknesses that matter for deployment. Full list and current status in Section 9.1. Afterwards, on request, items 1 (partly), 2, 20 and 21 were fixed: password change now revokes other sessions; Next.js upgraded to 16.4.3; Vite updated to 8.3.5. Item 1 (sign-up email enumeration) could only be fully fixed with email verification, which was added later in production mode.

7.11 Other requests

- A request to simulate 100 users was not carried out (the response was blocked by a safety system); it is not part of the project.
- A guide to running the project manually and the location of local account credentials was provided (Section 5).

7.12 Cloudflare security-audit skill (2026-10-30)

Cloned and installed `github.com/cloudflare/security-audit-skill` to `C:\Users\91635\.claude\skills\security-audit\` and ran a "quick full audit" (source review only; this Windows laptop has no sandbox, so nothing could be "confirmed" and the run is marked incomplete; the skill's validators do not run on Windows). Results:

- No confirmed vulnerabilities.
- Needs validation: (1) PDF decompression could exhaust server memory (later fixed, Section 7.15); (2) one member can permanently use up a workspace's 1,000-run and 2,000-approval lifetime quotas (still open; product decision).
- Rejected after verification: a suspected race between editing an approval and removing a member (the database serializes them).
- 3 areas deferred to a later audit: invitation lifecycle, procedures/skills/runs visibility, server-rendered page authorization.
- About 25 hardening notes (Section 9.3).

Reports: `REPORT.md`, `NEEDS-VALIDATION.md`, `FINDINGS-DETAIL.md`, `findings.json` in `C:\Users\91635\security-audit-skill\companyos\run-1\`.

7.13 Project moved (2026-11-01)

All files (32,608) were moved with their database and secrets to `C:\Users\91635\OneDrive\Desktop\CODE\COMPANYOS`; typecheck and unit tests were verified there; the old copy was removed (non-rebuildable parts sent to the Recycle Bin). From then on, all work happens only in the new folder.

7.14 GitHub (2026-11-04)

Initialized git, checked that no secrets were included (167 files; `.env.local`, `work/`, `node_modules/` excluded), and pushed to `https://github.com/sidsahoo1203/CompanyOS`. The first push hit a temporary GitHub "Internal Server Error"; retrying worked.

Commits so far: "Initial commit: CompanyOS milestones 1-4"; "Production mode for Render: verified sign-up, password reset, safer PDF parsing"; "Fix redeploy: only update passwords of existing database roles".

7.15 Making it production-ready (2026-11-04)

Decisions made by the owner: host on **Render only** (app + database), email through **Resend**, **open sign-up with verified email**.

Changes:

1. `src/lib/env.ts`: new `APP_MODE=production` (requires an HTTPS public origin, Resend settings, restricted roles; allows a hosted database). Local mode unchanged.
2. `src/lib/auth.ts`: email verification required in production; verification and reset emails; reset revokes all sessions; existing-email sign-up attempts email the owner instead of revealing the account; secure cookies in production; CSRF and origin checks pinned on; new rate limits for reset and verification endpoints.
3. `src/lib/email.ts`: sends through the Resend API (no extra package); escapes names; local mode prints the email to the terminal instead.
4. `src/lib/client-ip.ts`: production rate limits use the first `X-Forwarded-For` address set by Render.
5. New pages: forgot password and reset password; "Forgot password?" link; "Check your inbox" screen after sign-up; "Local development" labels shown only locally (`src/lib/mode.ts`).
6. `/api/health` endpoint for Render's health check.
7. PDF safety fix: parsing moved from a worker thread to a separate process; decompressed PDF data capped at 64 MB (a test with an 80 MB "zip bomb" PDF proves it stops without affecting the server); at most 2 parsers at once.
8. `scripts/db-setup-roles.mjs` (creates the restricted roles), `scripts/migrate.mjs` (allows the hosted database in production), `scripts/render-start.mjs` (roles → migrations → start the app on `0.0.0.0` without owner credentials).
9. HSTS header added at request time in `src/proxy.ts`.
10. `render.yaml` Blueprint, `.env.example` production section, `docs/deployment-render.md`, README updates.
11. Tests: env production rules, auth email/reset configuration, client-IP handling, password and verification UI, compressed-PDF and zip-bomb tests. Results: 439 unit tests and 89 integration tests passed; build passed; a production rehearsal on a fresh empty database applied all 10 migrations and passed health, page, cross-site and HSTS checks.

7.16 Deploying to Render (2026-11-04)

- The owner chose Render's free address (no own domain yet) and the free web plan.
- The service was created through "New Web Service" (manual) rather than the Blueprint, with: Build `npm ci && npm run build`, Start `npm run start:render`, Health check `/api/health`, region Singapore, and the environment variables in Section 6.5. The database `companyos-db` was created first and its Internal URL used as `MIGRATION_DATABASE_URL`.
- First deploy succeeded (roles created, migrations applied).
- Problem 1: sign-in and sign-up showed **"Invalid origin"** because Render assigned `companyos-bxk6.onrender.com` while `BETTER_AUTH_URL` was `https://companyos.onrender.com`. Fix: set `BETTER_AUTH_URL=https://companyos-bxk6.onrender.com` and redeploy.
- Problem 2: the sample account `founder@studio.example` does not exist on Render (local only). Fix: sign up with your own email (the same one used for Resend).

- Problem 3: the redeploy failed with "The database owner may not create roles". Cause: Render's database owner is not a superuser and may not re-state role attributes on existing roles. Fix (commit "Fix redeploy…"): existing roles now only get their password updated, and are checked to be unprivileged. Verified locally with a simulated non-superuser owner.
- Next step at the time of writing: push the fix, confirm `BETTER_AUTH_URL`, wait for "Live", then sign up with the Resend email and verify.

8. Deployment on Render: how it works and how to redo it

8.1 Settings of the web service

Field	Value
Runtime	Node; region Singapore; branch `main`; root directory empty
Build command	`npm ci && npm run build`
Start command	`npm run start:render`
Health check path	`/api/health`
Plan	Free (sleeps after 15 minutes idle; first visit then takes about a minute). Starter ($7/month) is always on.

A Blueprint alternative is in `render.yaml` (New → Blueprint), which creates the service and database together (it uses the paid `starter` web plan).

8.2 What happens on every deploy

1. `npm ci && npm run build` compiles the app.
2. `npm run start:render` runs `db-setup-roles.mjs` (creates or re-keys `companyos_app` and `companyos_auth`), then `migrate.mjs` (applies any new migrations; prints "Already applied" for old ones), then starts Next.js on `0.0.0.0:$PORT`.
3. A healthy log ends with `OK Ready in …` and the dashboard shows **Live**.

8.3 Checking it

1. `https://companyos-bxk6.onrender.com/api/health` → `{"status":"ok"}`.
2. Sign up with the email used for the Resend account; open the verification email; land on onboarding.
3. Test Forgot password.

8.4 Email limitation without a domain

Without a verified domain, Resend only delivers to the Resend account's own address, so only the owner can sign up. For customers: buy a domain (about ₹1,000 a year), verify it in Resend (DNS records), set `EMAIL_FROM=CompanyOS <no-reply@yourdomain>`, optionally add `app.yourdomain` as a Render custom domain and set `BETTER_AUTH_URL` to it.

8.5 Troubleshooting

Symptom	Cause and fix
"Invalid origin" on sign-in/sign-up	`BETTER_AUTH_URL` differs from the browser address. Set it exactly (https, no trailing slash) and redeploy.
`CompanyOS configuration is missing`	A required variable is empty or malformed (`RESEND_API_KEY` must start with `re_`).
`Production requires BETTER_AUTH_URL to be the public https:// origin`	Remove trailing slash/path; use https.
`The database owner may not create/update role …`	Provider restriction; create or update the two roles with the provider's tools, then redeploy.
`Applied migration changed`	An already-applied migration file was edited; restore it and add a new migration.
No verification email	Resend domain not verified, or signing up with an address other than the Resend account email. Check Resend → Emails.
Site slow to open	Free plan was sleeping; normal.

Symptom	Cause and fix
Free database disappeared	Free Render databases expire after about 30 days; use a paid plan for real data.

8.6 Why not Vercel (decision record)

The app is one Next.js application whose server pages read the database directly and whose API relies on same-origin cookies and checks. A Vercel frontend plus Render backend would break sign-in and every write. Options considered: everything on Render (chosen), whole app on Vercel with a separate database (needs code changes, parser and timeout issues), or Vercel as a proxy in front of Render (no benefit). Antideploy.com was also considered and not chosen; Render is the documented and tested path.

9. Security status

9.1 Penetration-test findings and current status

#	Finding	Severity	Status
1	Sign-up revealed which emails have accounts	Medium	Fixed in production (verified sign-up gives the same answer)
2	Password change did not log out other sessions	High	Fixed
3	Session cookies lacked Secure flag	Low locally	Fixed in production (secure cookies, HTTPS)
4	Email ownership never verified	Critical for public use	Fixed in production
5	No password reset	High	Fixed (reset by email)
6	One shared sign-in rate-limit bucket for everyone	Critical if exposed	Fixed in production (per client address)
7	No per-account brute-force protection; no two-factor login	Medium	Open
8	7-day sliding sessions, no idle timeout, no session list	Low	Open
9	Owner can approve own content	Medium (by design)	Open by design
10	Removed member's private sources unreadable to everyone	Medium	Open (needs ownership transfer)
11	No ownership transfer	Medium	Open
12	Members see former members' names/emails in records	Low	Accepted trade-off
13	No real antivirus	Critical for public uploads	Open
14	PDF/Word parsed inside the web server process	High	Fixed (separate process, no secrets)
15	DOCX zip-bomb check trusts declared sizes	Medium	Partly (memory bounded; CPU bounded by 15 s timeout)
16	Prompt-injection detection is a simple pattern	Low now	Open (must be redesigned before adding an AI model)
17	No overall quotas (sign-ups × workspaces × storage)	High if exposed	Open
18	Parsing on web server CPU, no global cap	Medium	Partly fixed (max 2 parsers per server)
19	Rate limits per person; more accounts = more allowance	Medium	Open
20	Next.js advisories	High	Fixed (16.4.3)
21	Vite advisories (test tool)	Low	Fixed (8.3.5)
22	No HTTPS/HSTS/database SSL	Critical if deployed	Fixed in production (Render HTTPS + HSTS; database on Render's private network)
23	Plain-text local secret files	Medium	Local only; production secrets live in Render settings
24	Weak monitoring; logins not audited	Medium	Open
25	No backups	Critical for real use	Open (use a paid database plan with backups)

#	Finding	Severity	Status
26	CSP allows inline styles	Low	Open

What held up in testing: workspace isolation (including direct database queries), role checks in app and database, invitation tokens, the approval gate (fingerprints, replay protection, single execution, stale refusal), origin and cross-site checks, input validation, tampered cookies, privileged sign-up fields, and inert rendering of uploaded text.

9.2 Security-audit leads

Lead	Status
PDF decompression could exhaust shared server memory	Fixed (separate process + 64 MB cap + test)
One member can permanently exhaust the 1,000-run / 2,000-approval workspace quota	Open: decide monthly or per-plan quotas
Edit-versus-removal approval race	Rejected (not a vulnerability)

9.3 Hardening suggestions not yet done

- Absolute maximum session lifetime; server-side length/character policy for display names.
- Wider invisible-character denylist (U+00AD, U+034F, U+061C, U+115F, U+3164, U+180E, U+2028, U+2029) or an allowlist; homoglyph-safe recipient domains.
- Block self-approval by anyone who authored any revision; re-check requester membership and source archive state in `decide_approval`.
- DOCX: reject entries whose actual size exceeds the declared size.
- Per-actor or global limits across workspaces; boundary-burst-safe rate windows.
- Check authorization before taking workspace row locks; pin `active_workspace` in a few older functions.
- Clear the invitation token from the URL immediately; replace `'unsafe-inline'` styles with nonces.
- Regular `npm audit`; consider replacing the beta `embedded-postgres` dev dependency.

10. Testing and quality commands

Command	Purpose
`npm run format:check` / `npm run format`	Prettier formatting
`npm run lint`	ESLint (TypeScript, React Hooks, accessibility)
`npm run typecheck`	Route types + strict TypeScript
`npm run test:unit`	439 unit tests (no database)
`npm run db:migrate -- --test` then `npm run test:integration`	89 PostgreSQL tests (database helper running)
`npm run test:coverage`	Unit coverage report in `coverage/`
`npm exec -- playwright install chromium`, `npm run build`, `npm run test:e2e`	Browser tests on port 3100
`npm run build`	Production build
`npm audit`	Dependency advisories

Definition of done (from AGENTS.md): plan first; all of the above pass; new behavior has unit, PostgreSQL and browser tests including denial cases; security, isolation, secrets, mobile and keyboard reviewed; README and verification docs updated; never claim results that were not run.

11. Open items and roadmap

Before paying customers:

1. Buy a domain and verify it in Resend (otherwise only the owner receives emails).
2. Upgrade the database to a paid plan with backups and test a restore; consider the Starter web plan (no sleeping).
3. Decide run/approval quotas (monthly or plan-based instead of lifetime).
4. Terms of service and privacy policy (customers upload business documents; India's DPDP Act, GDPR).
5. Account deletion, data export, workspace deletion, retention enforcement.
6. Real antivirus scanner; log shipping, uptime alerts, login audit.
7. Ownership transfer; two-factor login; session management screen.
8. Global quotas against abuse.
9. Follow-up security audit of the 3 deferred areas.

Product roadmap:

- **Milestone 5:** real integrations (Google Drive → Slack → Gmail) behind the same approval gate, with least-privilege scopes, revocation and retries.
- **Milestone 6:** usage tracking, monitoring, privacy workflows, billing and plans.
- Optional later: an AI model provider for drafting (requires explicit approval, budgets and privacy review), semantic search (pgvector).

12. Accounts, secrets and where things are

Thing	Where
Local sample accounts' password	`.env.local` → `SEED_PASSWORD`
Local database passwords	`work/local-db.json` and `.env.local` (never share)
Your live CompanyOS account	Created by signing up on the Render site; stored in the Render database
Production secrets	Render → companyos → Environment
Resend API key	Resend dashboard (shown once at creation; create a new one if lost)
Git commit email	Global git config (`git config --global user.email`)
Security audit outputs	`C:\Users\91635\security-audit-skill\companyos\run-1\`
Installed audit skill	`C:\Users\91635\.claude\skills\security-audit\`

Passwords in the app are stored only as salted hashes and cannot be read back; they can only be reset.

13. Glossary

Term	Meaning
Workspace	One company/team space; data never crosses workspaces
Source	A folder of documents with a visibility rule
Chunk / passage	A piece of extracted document text used for search and citations
Procedure	Instructions plus required sources; versioned
Skill	A runnable, pinned procedure version with contract, budget and timeout
Run	One execution of a skill producing a draft and proposals
Approval request	A snapshot of a proposed action awaiting an owner/admin decision
Fingerprint (digest)	SHA-256 of the exact content reviewed; must match to approve
Local outbox	Database table recording approved actions (nothing is sent)
RLS	PostgreSQL row-level security, filtering rows per workspace
SECURITY DEFINER function	Narrow SQL function that performs a write after its own checks
APP_MODE	`local` (laptop) or `production` (hosted)
Blueprint	Render's `render.yaml` that creates services from the repository

14. Starter prompt for a new Claude Code session

Copy, adjust the last line, and paste together with this handbook:

```
I'm continuing work on CompanyOS. The project folder is
C:\Users\91635\OneDrive\Desktop\CODE\COMPANYOS (work only there) and the repo
is github.com/sidsahoo1203/CompanyOS. Read the attached CompanyOS Project
Handbook first, then AGENTS.md and README.md in the repo. Follow AGENTS.md
rules: plan first, don't deploy or change production settings without my
approval, never print secrets from .env.local or work/, never edit applied
migrations. The app is deployed on Render at https://companyos-bxk6.onrender.com.
Today I want to: <describe the task>
```